



Sistemas Embebidos I
Otoño 2026

Examen 1: Duelo de reacción

Jonathan Hernández Lazcano

Eduardo Gabriel Prado Granados

Camila Rodríguez Rosas

Ana Karen Rojas Ledesma

Hamid Susen Santiago García

Lunes, 28 de septiembre de 2026

Índice

Índice.....	1
Examen 1: Duelo de reacción.....	2
Introducción.....	2
Descripción de la práctica.....	2
Objetivo.....	3
Marco teórico.....	3
Materiales.....	5
Esquemático.....	5
Desarrollo.....	5
Descripción general del sistema.....	5
Diagrama de conexiones y montaje.....	6
Diagrama de bloques: qué corre en cada ISR y qué corre en el bucle.....	8
Máquina de estados del juego.....	10
Fragmentos de código explicados.....	10
1. Configuración de botones, UART e interrupciones.....	10
2. ISR de UART y buffer circular de recepción.....	11
3. Transmisión no bloqueante.....	12
4. Consola: eco, armado de línea y comandos.....	13
5. Inicio de partida y tiempo aleatorio.....	15
6. Callback del timer.....	16
7. ISR de los botones: antirrebote y medición.....	17
8. Reporte del resultado y bucle principal.....	19
9. Programa de la Pico de apoyo: puente USB–UART.....	20
Pruebas de funcionamiento por consola UART.....	21
Verificación del tiempo de reacción con osciloscopio.....	25
Primera medición: modo Roll.....	25
Segunda medición: tiempo de reacción.....	26
Duración de la pulsación.....	27
Comparación con la consola UART.....	28
Conclusiones.....	28
Referencias.....	29

Examen 1: Duelo de reacción

Introducción

En este proyecto se desarrolla un duelo de reacción para dos jugadores utilizando una Raspberry Pi Pico, con el propósito de poner en práctica conceptos de sistemas embebidos. El juego utiliza dos botones y dos LEDs para representar los diferentes estados de la partida y determinar qué jugador reacciona primero ante la señal de inicio.

Para su funcionamiento se emplean GPIO, interrupciones, temporizadores y medición de tiempo, permitiendo detectar las pulsaciones de los jugadores y registrar su tiempo de reacción. También se incorpora una comunicación mediante UART, desde la cual se pueden iniciar partidas, configurar el periodo de parpadeo y consultar las estadísticas obtenidas.

El sistema busca que todas estas funciones trabajen de manera simultánea, utilizando interrupciones y temporizadores para evitar bloqueos en el programa principal y lograr una respuesta precisa durante cada partida.

Descripción de la práctica

La práctica consiste en desarrollar un juego de reflejos para dos jugadores utilizando una Raspberry Pi Pico. El sistema permite que ambos jugadores compitan para responder ante una señal luminosa, registrando cuál de ellos reacciona primero y el tiempo empleado en hacerlo.

Para lograr el funcionamiento del juego se integran diferentes recursos de la plataforma, como entradas y salidas GPIO, temporización, interrupciones y comunicación serial mediante UART. Estos elementos permiten que el sistema responda a las acciones de los jugadores y mantenga el funcionamiento de la partida de manera concurrente.

Además del funcionamiento del juego, la práctica contempla la consulta de los resultados mediante la consola serial y la comprobación experimental de la medición del tiempo de reacción utilizando un osciloscopio. De esta manera, se busca

relacionar la implementación del sistema embebido con una medición física que permita verificar su funcionamiento.

Objetivo

Desarrollar e implementar un juego de reflejos para dos jugadores utilizando una Raspberry Pi Pico, integrando GPIO, temporizadores, interrupciones y comunicación UART para coordinar de manera concurrente el control de los LEDs, la detección de las pulsaciones y la interacción con la consola serial.

El sistema deberá identificar correctamente los estados de la partida, determinar al jugador ganador o una salida en falso, registrar el tiempo de reacción mediante marcas de tiempo y almacenar el resultado de cada partida para su posterior consulta.

Finalmente, comprobar el funcionamiento y la precisión del sistema mediante una medición con osciloscopio, comparando el tiempo de reacción obtenido físicamente con el valor reportado por la consola UART.

Marco teórico

Interrupciones

Una interrupción permite que el microcontrolador responda a un evento sin tener que estar consultando continuamente el estado de un dispositivo mediante *polling*. En este caso, las interrupciones de GPIO permiten detectar directamente cuándo un jugador presiona su botón.

El SDK permite configurar interrupciones para eventos como flancos de subida y de bajada mediante constantes como GPIO_IRQ_EDGE_RISE y GPIO_IRQ_EDGE_FALL. También proporciona una función de *callback* que recibe el GPIO que generó la interrupción y el evento ocurrido.

Para el examen, cada botón está conectado a un GPIO de entrada y es atendido mediante una interrupción. La rutina de servicio de interrupción (ISR) identifica qué botón fue presionado, obtiene una marca de tiempo en microsegundos y determina si la pulsación corresponde a una respuesta válida, una salida en falso o una pulsación que debe ignorarse.

Temporizadores y medición del tiempo

Los temporizadores permiten ejecutar acciones relacionadas con el paso del tiempo sin tener que detener completamente la ejecución del programa. El SDK proporciona funciones para obtener marcas de tiempo, como `time_us_64()`, que permite obtener el contador de tiempo en microsegundos. Esta función resulta útil para medir intervalos con precisión.

Para esta práctica se utiliza un temporizador repetitivo mediante `add_repeating_timer_ms()`. Este mecanismo ejecuta periódicamente una función *callback*. Con un periodo negativo, el intervalo se mide desde el inicio de una ejecución del *callback* hasta el inicio de la siguiente, evitando que el tiempo de ejecución del propio *callback* se acumule al periodo. El temporizador puede detenerse mediante `cancel_repeating_timer()`.

En el juego, el mismo temporizador se utiliza para controlar el parpadeo del LED rojo durante el estado de preparación y para llevar el conteo del tiempo máximo de cinco segundos durante la ventana de respuesta.

Máquina de estados finitos FSM

Una máquina de estados finitos (FSM) es un modelo utilizado para representar sistemas que pueden encontrarse en un conjunto definido de estados y cambiar entre ellos cuando ocurre un determinado evento. Una FSM está compuesta principalmente por estados y transiciones. Los estados representan los diferentes modos en los que puede encontrarse el sistema, mientras que las transiciones indican las condiciones necesarias para pasar de un estado a otro. De esta manera se nos permite revisar continuamente diferentes entradas y mantener su propio estado y referencia temporal.

UART

La UART (Universal Asynchronous Receiver/Transmitter) es un periférico de comunicación serial que permite transmitir y recibir datos entre dispositivos mediante una comunicación asíncrona. En este tipo de comunicación, los datos se envían de manera secuencial, bit por bit, utilizando una línea de transmisión (TX) y una línea

de recepción (RX). La comunicación requiere que ambos dispositivos utilicen la misma configuración de velocidad y formato de los datos.

En este examen, se utiliza como interfaz entre la Raspberry Pi Pico y la consola serial para permitir la interacción con el juego. La comunicación se configura mediante UART por hardware a 115200 baudios, 8N1, y la recepción de datos se realiza mediante interrupciones y un buffer circular en memoria. Esto permite que la consola pueda recibir comandos mientras el resto del sistema continúa ejecutando el juego.

A través de la consola se pueden ejecutar comandos como start, para iniciar una partida; set periodo <ms>, para modificar el periodo de parpadeo del LED; y get stats, para consultar la configuración y el resultado de la última partida. También se utiliza el eco de los caracteres introducidos para mostrar en la consola lo que escribe el usuario.

De esta manera, UART permite controlar y consultar el sistema sin interferir con la ejecución de los botones, LEDs y temporizadores, formando parte de la interacción principal entre el usuario y el sistema embebido.

Materiales

- 1 LED y su resistencia limitadora, conectados al pin GP16 de la Pico principal.
- 2 cables USB tipo A a micro-USB, uno por tarjeta.
- Computadora con Visual Studio Code y la extensión *Raspberry Pi Pico*.
- Pico SDK 2.3.1 con la cadena de herramientas arm-none-eabi-gcc 13.2, CMake y Ninja.
- Monitor serial integrado de Visual Studio Code, configurado a 115200 baud con terminación de línea CRLF.
- Python 3 con la biblioteca pyserial, para el script generador de tráfico continuo.
- Lucidchart, para la elaboración de los diagramas de flujo.

Esquemático

Desarrollo

Descripción general del sistema

El sistema está formado por dos programas, cada uno en su propia Raspberry Pi Pico 2. **PICO_PRINCIPAL_EXAMEN** contiene toda la lógica del juego: botones, LEDs, temporizador, medición del tiempo de reacción y consola de comandos por UART0. **PICO_APOYO_EXAMEN** se ejecuta en una segunda tarjeta que no participa en el juego: solo reenvía bytes entre el puerto USB de la computadora y la UART de la Pico principal, como un intérprete que traduce entre dos personas que hablan idiomas distintos.

El juego se opera desde una consola serial. Al encenderse, envía por UART un mensaje de bienvenida con la lista de comandos. Una partida inicia con el comando start: el LED rojo comienza a parpadear con el periodo configurado y, después de un tiempo aleatorio entre 2 y 5 segundos, se apaga y se enciende el LED verde. El primer jugador que presiona su botón con el LED verde encendido gana y el sistema reporta su tiempo de reacción. Si un jugador presiona antes del verde comete una salida en falso y el punto es para su rival; si nadie presiona durante 5 segundos, la partida termina sin ganador.

Tabla 1. Comandos de la consola UART implementados.

Comando	Acción	Respuesta
start	Inicia una partida si no hay otra en curso	Listo o Error: partida en curso
set periodo <ms>	Cambia el periodo de parpadeo del LED rojo (50–1000 ms); solo con el juego detenido	Listo o Error: periodo fuera de rango (50-1000 ms)
get stats	Muestra periodo, resultado de la última partida, errores de UART y marcador	Bloque de estadísticas
reset	Reinicia el marcador de ambos jugadores	Listo
Otro texto	—	Error: comando desconocido

Diagrama de conexiones y montaje

La Pico principal se comunica con la computadora a través de la Pico de apoyo, que funciona como puente USB–UART. Las líneas se conectan cruzadas: el GP0 (TX) de la principal va al GP1 (RX) de la de apoyo y el GP0 (TX) de la de apoyo va al GP1 (RX) de la principal, ya que lo que una transmite la otra debe recibirlo; además, ambas tarjetas comparten tierra para tener la misma referencia de voltaje. Cada Pico se alimenta con su propio cable USB, pero solo la de apoyo se usa como puerto serial en la computadora. Los LEDs se conectan en configuración de fuente de corriente (el GPIO entrega corriente al LED cuando está en alto) y los botones se conectan entre el GPIO y tierra, aprovechando la resistencia pull-up interna; por ello, en reposo el pin lee 1 y al presionar se produce un flanco de bajada.

Tabla 2. Asignación de pines de la Pico principal.

Pin	Función	Configuración
-----	---------	---------------

GP0	UART0 TX	GPIO_FUNC_UART
GP1	UART0 RX	GPIO_FUNC_UART
GP16	LED rojo (preparados)	Salida, activo en alto
GP17	LED verde (ya)	Salida, activo en alto
GP18	Botón jugador 1	Entrada con pull-up, IRQ flanco de bajada
GP19	Botón jugador 2	Entrada con pull-up, IRQ flanco de bajada

Tabla 3. Conexiones entre la Pico principal y la Pico de apoyo.

Pico principal	Pico de apoyo	Señal
GP0 (UART0 TX)	GP1 (UART0 RX)	Respuestas y eco hacia la consola
GP1 (UART0 RX)	GP0 (UART0 TX)	Comandos escritos en la consola
GND	GND	Referencia común
—	USB	Puerto serial virtual hacia la computadora

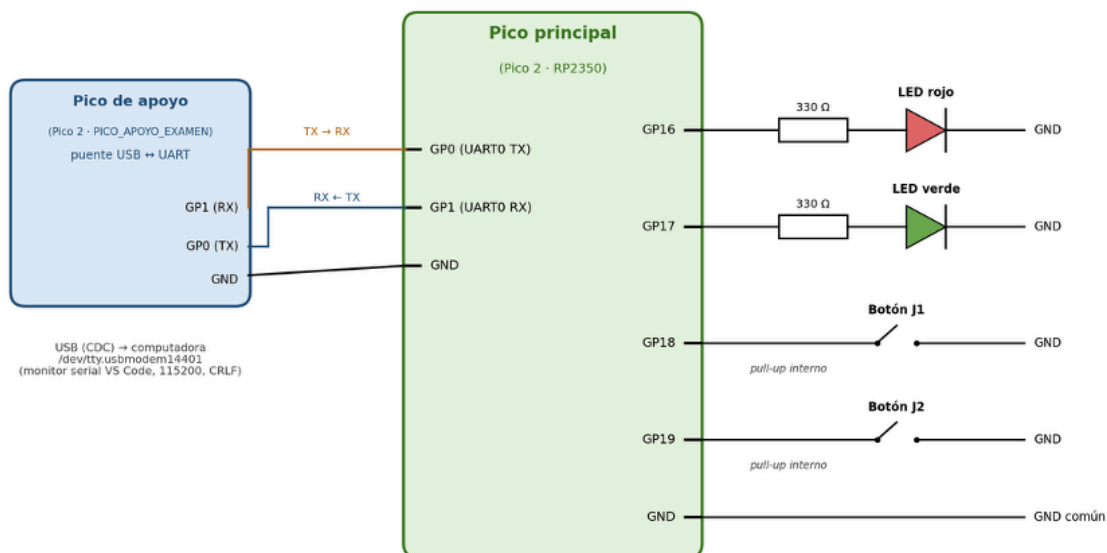


Figura 1. Diagrama de conexiones de la Pico principal, la Pico de apoyo, los LEDs y los botones.

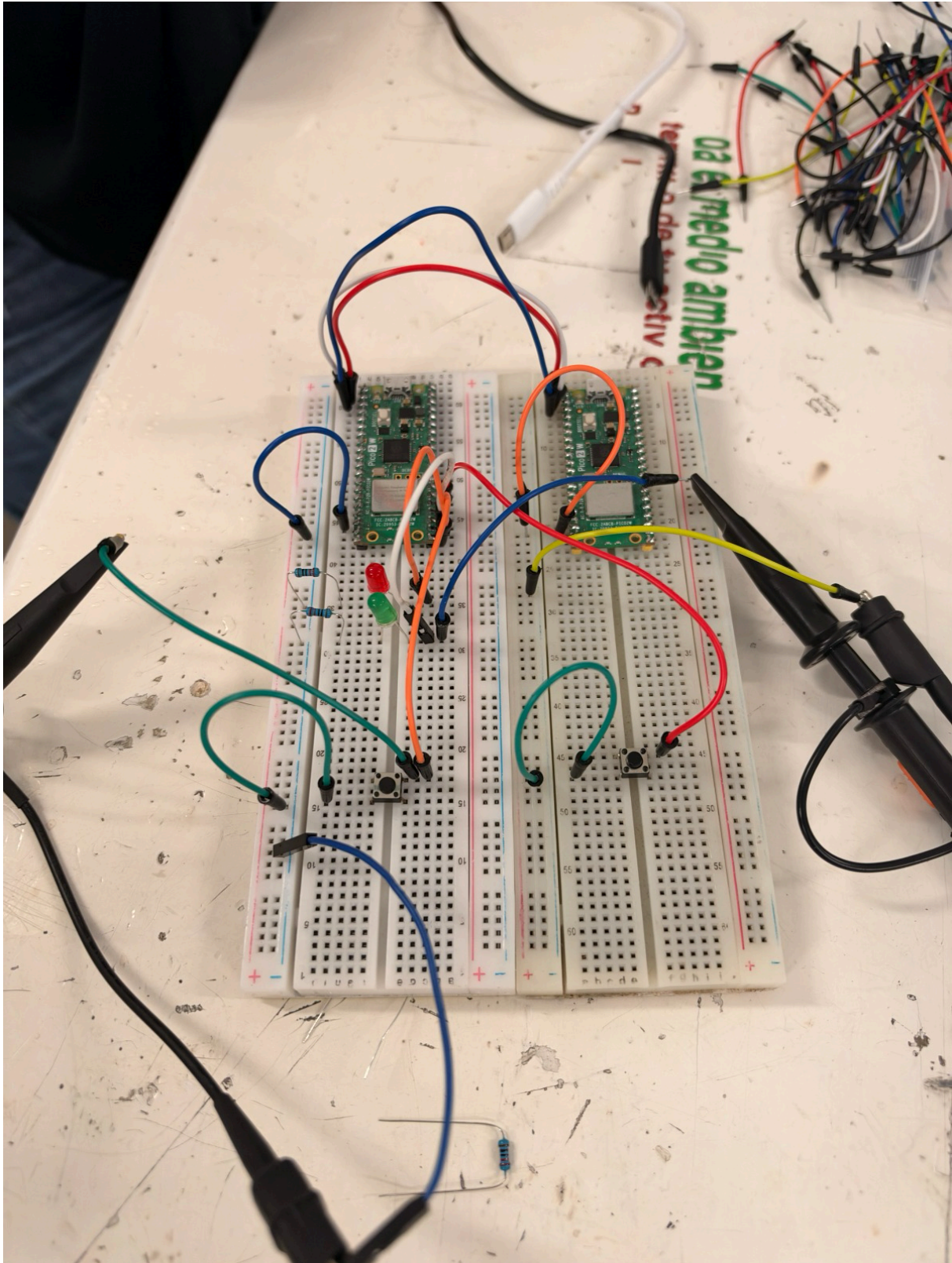


Figura 2. Fotografía del montaje en protoboard.

Diagrama de bloques: qué corre en cada ISR y qué corre en el bucle

La arquitectura del programa sigue el modelo *foreground/background*: tres rutinas de interrupción atienden los eventos de hardware en cuanto ocurren y dejan su resultado en

buffers o banderas, mientras que el bucle principal consume esa información sin bloquearse. La Figura 3 muestra esta división y la Tabla 4 la resume.

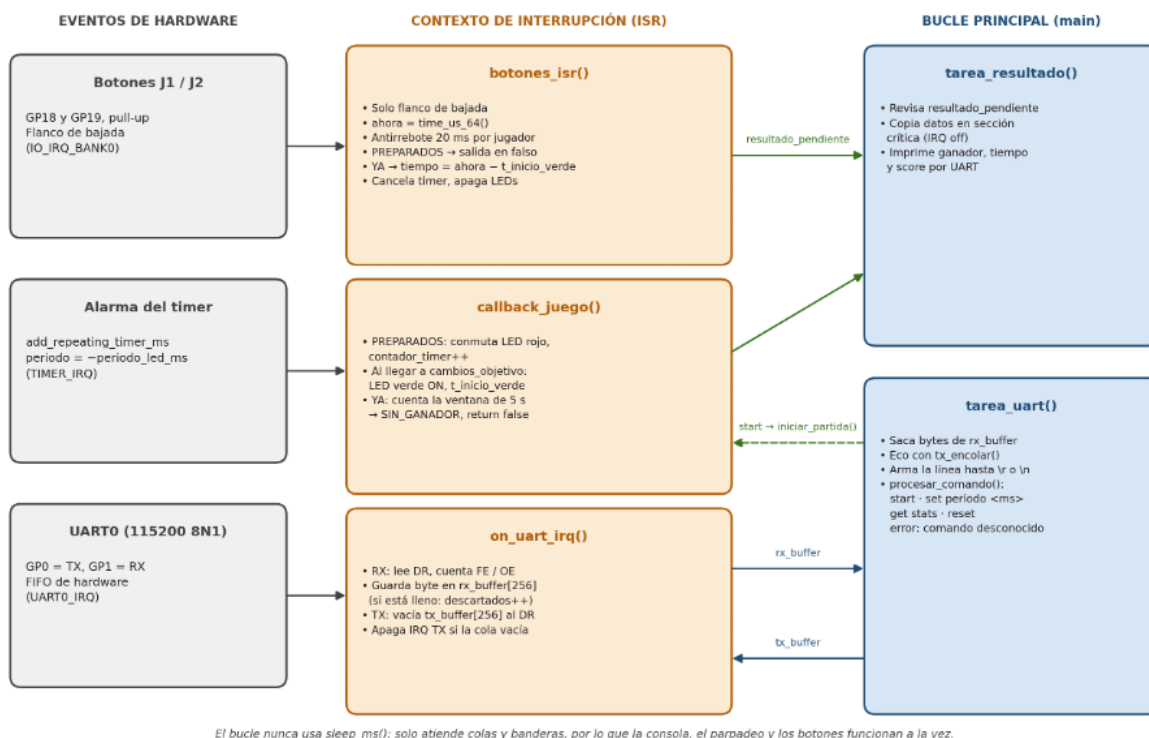


Figura 3. Diagrama de bloques del sistema con las tareas que se ejecutan en cada ISR y en el bucle principal.

Tabla 4. Distribución de responsabilidades entre las ISR y el bucle principal.

Contexto	Se dispara por	Trabajo que realiza
botones_isr()	Flanco de bajada en GP18 o GP19	Marca de tiempo, antirrebote, decide GANA o FALSO, detiene el timer, actualiza el marcador y levanta resultado_pendiente
callback_juego()	Vencimiento de la alarma del timer repetitivo	Parpadeo del LED rojo, cuenta del tiempo aleatorio, encendido del LED verde con su marca de tiempo y cierre por los 5 s sin respuesta
on_uart_irq()	Byte recibido o espacio libre en la FIFO de TX de UART0	Copia bytes de la FIFO a rx_buffer, cuenta errores y llena la FIFO de TX desde tx_buffer
tarea_uart() (bucle)	Hay bytes en rx_buffer	Eco, armado de la línea e interpretación de comandos
tarea_resultado() (bucle)	resultado_pendiente == true	Formatea e imprime el resultado y el marcador

Ninguna ISR llama a `printf()` ni a `snprintf()`: el formateo de texto, que es la operación más lenta, se realiza siempre en el bucle principal. Tampoco se utiliza `sleep_ms()` en ninguna parte del programa, por lo que los tres flujos (consola, parpadeo y botones) avanzan al mismo tiempo.

El diagrama solo incluye la Pico principal, que es donde se ejecuta el juego. La Pico de apoyo queda fuera del diagrama porque no maneja interrupciones propias: su programa es un único bucle que revisa el USB y la UART por *polling* y reenvía lo que encuentra, como se explica en el Fragmento 13.

Máquina de estados del juego

El estado del juego se guarda en la variable `estado_juego`, y las transiciones las provocan tres fuentes distintas: la consola (start), el timer (fin del tiempo aleatorio o de la ventana de 5 s) y los botones.



Figura 4. Máquina de estados del duelo de reacción con sus tres estados y transiciones.

Fragmentos de código explicados

1. Configuración de botones, UART e interrupciones

Fragmento 1. Inicialización en `main()` (líneas 687–717 de `PICO_PRINCIPAL_EXAMEN.c`)

```
int main(void) {

    // LEDs
    gpio_init(LED_ROJO);
    gpio_set_dir(LED_ROJO,GPIO_OUT);
    gpio_put(LED_ROJO,0);
    gpio_init(LED_VERDE);
    gpio_set_dir(LED_VERDE,GPIO_OUT);
    gpio_put(LED_VERDE,0);

    // Botones
    gpio_init(BOTON_J1);
    gpio_set_dir(BOTON_J1,GPIO_IN);
    gpio_pull_up(BOTON_J1);
    gpio_init(BOTON_J2);
    gpio_set_dir(BOTON_J2,GPIO_IN);
    gpio_pull_up(BOTON_J2);

    // Interrupcion por botones con callback
    gpio_set_irq_enabled_with_callback(
        BOTON_J1,
        GPIO_IRQ_EDGE_FALL,
        true,
        &botones_isr
    );

    // Habilitar interrupción por edge fall
    gpio_set_irq_enabled(BOTON_J2,GPIO_IRQ_EDGE_FALL,true);

    // Set UART
    gpio_set_function(TX_PIN,GPIO_FUNC_UART);
    gpio_set_function(RX_PIN,GPIO_FUNC_UART);

    // Inicialización UART
    uart_init(UART_ID,BAUD_RATE);

    // Configuración 8N1
    uart_set_format(UART_ID,8,1,UART_PARITY_NONE);

    // FIFO UART habilitada
    uart_set_fifo_enabled(UART_ID,true);

    // Interrupciones UART
    irq_set_exclusive_handler(UART0_IRQ,on_uart_irq);
    irq_set_enabled(UART0_IRQ,true);
    uart_set_irq_enables(UART_ID,true,false);
}
```

Los botones se configuran como entradas con pull-up interno y con interrupción por flanco de bajada. `gpio_set_irq_enabled_with_callback()` registra `botones_isr` como el *callback* único del banco de GPIO; para el segundo botón basta con `gpio_set_irq_enabled()`, ya que comparte el mismo *callback*. Después se asignan GP0 y GP1 a la función UART, se inicializa UART0 a 115200 baudios con formato 8N1 y se habilita su FIFO de hardware. Por último se registra `on_uart_irq` como manejador exclusivo de `UART0_IRQ`, se habilita la interrupción en el NVIC y, en el periférico, solo la interrupción de recepción (`uart_set_irq_enables(UART_ID, true, false)`); la de transmisión se enciende únicamente cuando hay datos que enviar.

2. ISR de UART y buffer circular de recepción

Fragmento 2. Rutina de interrupción de UART0 (líneas 159–205 de PICO_PRINCIPAL_EXAMEN.c)

```
static void on_uart_irq(void) {

    // Recepción
    while (uart_is_readable(UART_ID)) {

        // Leer registro de datos UART
        uint32_t dr = uart_get_hw(UART_ID)->dr;

        // Revisar framing error
        if (dr & UART_UARTDR_FE_BITS) {
            framing_errors++;
        }

        // Revisar overrun
        if (dr & UART_UARTDR_OE_BITS) {
            overruns++;
        }

        // Obtener únicamente los 8 bits del dato
        uint8_t c = (uint8_t)(dr & 0xFFu);

        // Calcular siguiente posición
        uint16_t siguiente = (rx_head + 1) % RX_BUF_SIZE;

        // Si existe espacio
        if (siguiente != rx_tail) {
            rx_buffer[rx_head] = c;
            rx_head = siguiente;
        }

        // Buffer circular lleno
        else {
            descartados++;
        }
    }

    // Transmisión
    while (uart_is_writable(UART_ID) && tx_tail != tx_head) {
        uart_get_hw(UART_ID)->dr = tx_buffer[tx_tail];
        tx_tail = (tx_tail + 1) % TX_BUF_SIZE;
    }

    // Si ya no queda nada por transmitir
    if (tx_tail == tx_head) {
        uart_set_irq_enables(UART_ID,true,false);
    }
}
```

En la parte de recepción, la ISR vacía la FIFO de hardware mientras `uart_is_readable()` sea verdadero. Cada lectura del registro DR entrega el byte en los 8 bits menos significativos y, en los bits superiores, las banderas de error; por eso se revisan `UART_UARTDR_FE_BITS` y `UART_UARTDR_OE_BITS` para contar *framing errors* y *overruns* antes de quedarse con `dr & 0xFF`. El byte se guarda en `rx_buffer` solo si hay espacio; si el buffer de software está lleno, se incrementa `descartados` en lugar de sobrescribir datos. En la parte de transmisión, la ISR copia bytes de `tx_buffer` al registro de datos mientras la FIFO de TX tenga espacio y, cuando la cola queda vacía, deshabilita la interrupción de TX para no entrar en un ciclo infinito de interrupciones.

3. Transmisión no bloqueante

Fragmento 3. Función `tx_encolar()` (líneas 113–148 de `PICO_PRINCIPAL_EXAMEN.c`)

```
static void tx_encolar(uint8_t c) {

    // Evitar que la ISR UART modifique tx_tail mientras estamos actualizando el buffer
    uint32_t estado_irq = save_and_disable_interrupts();

    // Saber si la cola estaba vacía antes de agregar el dato
    bool cola_vacia = (tx_head == tx_tail);

    // Calcular siguiente posición del buffer circular
    uint16_t siguiente = (tx_head + 1) % TX_BUF_SIZE;

    // Si siguiente != tail, existe espacio
    if (siguiente != tx_tail) {

        // Guardar carácter
        tx_buffer[tx_head] = c;

        // Avanzar head
        tx_head = siguiente;

        // Si la cola estaba completamente vacía,
        // mandamos directamente el primer byte al UART.
        if (cola_vacia && uart_is_writable(UART_ID)) {
            uart_get_hw(UART_ID)->dr = tx_buffer[tx_tail];
            tx_tail = (tx_tail + 1) % TX_BUF_SIZE;
        }

        // Si quedan datos habilitar la IRQ de TX
        if (tx_tail != tx_head) {
            uart_set_irq_enables(UART_ID, true, true);
        }
        else {
            uart_set_irq_enables(UART_ID, true, false);
        }
    }

    // Restaurar interrupciones
    restore_interrupts(estado_irq);
}
```

Toda la salida por UART pasa por esta función, que coloca el carácter en tx_buffer en lugar de esperar a que la UART termine de enviarlo; así, imprimir un mensaje largo no detiene el programa. Como tanto el bucle principal como la ISR modifican tx_tail, la actualización se protege con save_and_disable_interrupts(). Si la cola estaba vacía se escribe el primer byte directamente en el registro de datos para "arrancar" la transmisión, y después se habilita la interrupción de TX solo si quedan bytes pendientes. Si el buffer está lleno, el carácter se descarta en lugar de bloquear.

4. Consola: eco, armado de línea y comandos

Fragmento 4. Tarea de consola en el bucle principal (líneas 646–675 de PICO_PRINCIPAL_EXAMEN.c)

```
static void tarea_uart(void) {

    // Vaciar completamente el buffer RX
    while (rx_tail != rx_head) {
        uint8_t c = rx_buffer[rx_tail];
        rx_tail = (rx_tail + 1) % RX_BUF_SIZE;

        // Eco
        tx_encolar(c);

        // Fin de línea
        if (c == '\r' || c == '\n') {
            linea[n] = '\0';

            // Evitar procesar línea vacía
            if (n > 0) {
                procesar_comando(linea);
            }

            n = 0;
        }

        // Seguir acumulando caracteres
        else {
            if (n < LINE_SIZE - 1) {
                linea[n] = (char)c;
                n++;
            }
        }
    }
}
```

tarea_uart() consume todos los bytes disponibles en rx_buffer. Cada carácter se reenvía como eco para que el usuario vea lo que escribe y se acumula en linea, con un límite de 63 caracteres para no desbordar el arreglo. Al recibir \r o \n se cierra la cadena con \0 y se envía a procesar_comando(); las líneas vacías se ignoran, lo que evita procesar dos veces un fin de línea CRLF.

Fragmento 5. Comando set periodo (líneas 608–640 de PICO_PRINCIPAL_EXAMEN.c)

```
if (strncmp(cmd, "set periodo", 11) == 0) {

    // Candado: partida en curso
    if (estado_juego != DETENIDO) {
        tx_encolar_texto(" Error: partida en curso\r\n");
        return;
    }

    // Asignar valor del periodo
    unsigned int nuevo_periodo;
    char extra;

    int encontrados =
        sscanf(
            cmd,
            "set periodo %u %c",
            &nuevo_periodo,
            &extra
        );

    if (encontrados == 1) {

        // Comprobar rango 50–1000 ms
        if (nuevo_periodo >= 50 && nuevo_periodo <= 1000) {
            periodo_led_ms = nuevo_periodo;
            tx_encolar_texto(" Listo\r\n");
        }
        else {
            tx_encolar_texto(
                " Error: periodo fuera de rango (50–1000 ms)\r\n"
            );
        }
    }
    else {
        tx_encolar_texto(" Error: comando desconocido\r\n");
    }

    return;
}
```

El comando se reconoce con `strncmp()` y el valor se extrae con `sscanf()`. El especificador adicional `%c` detecta texto sobrante después del número: solo si `sscanf()` devuelve exactamente 1 el comando está bien formado. El periodo se valida en el rango de 50 a 1000 ms y no puede cambiarse con una partida en curso, ya que el timer ya se configuró con el valor anterior.

5. Inicio de partida y tiempo aleatorio

Fragmento 6. Cálculo del tiempo aleatorio en `iniciar_partida()` (líneas 294–314 de `PICO_PRINCIPAL_EXAMEN.c`)

```

periodo_timer_actual_ms = periodo_led_ms;

// Calcular el mínimo de callbacks equivalente a 2 s.
uint32_t min_cambios =
    (ESPERA_MIN_MS + periodo_timer_actual_ms - 1u)
    / periodo_timer_actual_ms;

// Calcular el máximo de callbacks equivalente a 5 s.
uint32_t max_cambios =
    ESPERA_MAX_MS / periodo_timer_actual_ms;

// Seguridad
if (max_cambios < min_cambios) {
    max_cambios = min_cambios;
}

// Tiempo aleatorio
cambios_objetivo =
    min_cambios +
    (rand() % (max_cambios - min_cambios + 1u));

contador_timer = 0;
estado_juego = PREPARADOS;

// Timer repetitivo
timer_activo = add_repeating_timer_ms(
    -(int32_t)periodo_timer_actual_ms,
    callback_juego,
    NULL,
    &timer_juego
);

```

El tiempo de espera antes del LED verde se expresa como un número entero de ejecuciones del *callback* del timer. Se calcula el mínimo de ejecuciones que cubre 2 s (redondeando hacia arriba) y el máximo que cabe en 5 s (redondeando hacia abajo), y se elige un valor aleatorio entre ambos con `rand()`. Con el periodo de 350 ms usado en las pruebas, el LED verde aparece después de 6 a 14 ejecuciones, es decir, entre 2.1 y 4.9 s. Finalmente se arma el timer con periodo negativo para que el intervalo entre ejecuciones no se alargue con la duración del *callback*.

6. Callback del timer

Fragmento 7. Estado PREPARADOS en `callback_juego()` (líneas 213–242 de `PICO_PRINCIPAL_EXAMEN.c`)

```

if (estado_juego == PREPARADOS) {

    // Invertir estado del LED rojo
    rojo_encendido = !rojo_encendido;
    gpio_put(LED_ROJO, rojo_encendido);

    // Contar una ejecución del timer
    contador_timer++;

    // Si se cumplió el tiempo aleatorio...
    if (contador_timer >= cambios_objetivo) {

        // Configuración LEDs
        rojo_encendido = false;
        gpio_put(LED_ROJO, 0);
        gpio_put(LED_VERDE, 1);

        // Tomar timestamp del inicio del verde
        t_inicio_verde = time_us_64();

        // Cambiar estado del juego
        estado_juego = YA;

        // Reiniciar contador para los 5 s
        contador_timer = 0;
    }

    // true significa que el timer continúa
    return true;
}

```

En el estado PREPARADOS, cada ejecución invierte el LED rojo y aumenta contador_timer. Cuando el contador alcanza cambios_objetivo, se apaga el rojo, se enciende el verde y en la línea siguiente se toma t_inicio_verde con time_us_64(); esta marca es el punto de partida de la medición del tiempo de reacción. El estado cambia a YA y el contador se reinicia para medir la ventana de respuesta.

Fragmento 8. Estado YA en `callback_juego()` (líneas 245–272 de `PICO_PRINCIPAL_EXAMEN.c`)

```
if (estado_juego == YA) {
    contador_timer++;

    // Calcular callbacks para representar 5s.
    uint32_t ciclos_5s =
        (VENTANA_VERDE_MS + periodo_timer_actual_ms - 1u)
        / periodo_timer_actual_ms;

    // Verde + de 5s.
    if (contador_timer >= ciclos_5s) {

        // Apagar LED verde
        gpio_put(LED_VERDE, 0);

        // Guardar resultado
        resultado = SIN_GANADOR;
        ganador = 0;
        jugador_falso = 0;
        tiempo_reaccion_us = 0;

        // Regresar al estado inicial
        estado_juego = DETENIDO;
        timer_activo = false;

        resultado_pendiente = true;

        // false detiene el repeating timer
        return false;
    }

    return true;
}
```

En el estado YA el mismo timer cuenta cuántas ejecuciones equivalen a 5 s. Si se alcanza ese número sin que nadie presione, se apaga el LED verde, se registra SIN_GANADOR y el *callback* devuelve false, lo que detiene el timer repetitivo. Dado que la ventana se mide en múltiplos del periodo, su duración real es de $\lceil 5000 / \text{periodo} \rceil \times \text{periodo}$; con 350 ms resultan 15 ejecuciones, es decir, 5.25 s.

7. ISR de los botones: antirrebote y medición

Fragmento 9. Identificación del botón y antirrebote (líneas 329–351 de PICO_PRINCIPAL_EXAMEN.c)

```
static void botones_isr(uint gpio, uint32_t events) {

    if (!(events & GPIO_IRQ_EDGE_FALL)) {
        return;
    }

    uint64_t ahora = time_us_64();

    uint8_t jugador;

    if (gpio == BOTON_J1) {
        jugador = 1;

        if (ultimo_j1_us != 0 &&
            ahora - ultimo_j1_us < DEBOUNCE_US) {
            return;
        }

        ultimo_j1_us = ahora;
    }

    else if (gpio == BOTON_J2) {
        jugador = 2;

        if (ultimo_j2_us != 0 &&
            ahora - ultimo_j2_us < DEBOUNCE_US) {
            return;
        }

        ultimo_j2_us = ahora;
    }

    else {
        return;
    }
}
```

Lo primero que hace la ISR es tomar la marca de tiempo ahora, para que el cálculo posterior no incluya el tiempo de las demás instrucciones. Después identifica al jugador por el número de GPIO y aplica el antirrebote: si han pasado menos de 20 ms desde la última pulsación aceptada de ese mismo botón, la interrupción se ignora. Cada jugador tiene su propio registro de tiempo, así que el rebote de un botón no bloquea al otro.

Fragmento 10. Respuesta válida en el estado YA (líneas 393–425 de PICO_PRINCIPAL_EXAMEN.c)

```

if (estado_juego == YA) {

    // Detener timer
    if (timer_activo) {
        cancel_repeating_timer(&timer_juego);
        timer_activo = false;
    }

    // Apagar LEDs
    gpio_put(LED_VERDE, 0);
    gpio_put(LED_ROJO, 0);
    rojo_encendido = false;

    // Guardar ganador
    ganador = jugador;
    jugador_falso = 0;

    // Medición del tiempo de reacción
    tiempo_reaccion_us = ahora - t_inicio_verde;
    resultado = GANA;

    // Actualizar score
    if (jugador == 1) {
        score_j1++;
    }
    else {
        score_j2++;
    }

    // Terminar partida
    estado_juego = DETENIDO;
    resultado_pendiente = true;
}

```

Si la pulsación llega en el estado YA, se cancela el timer para que no declare SIN_GANADOR, se apagan los LEDs, se registra al ganador y se calcula `tiempo_reaccion_us = ahora - t_inicio_verde`. Como el estado pasa inmediatamente a DETENIDO, la pulsación del segundo jugador que llegue después se ignora. El caso PREPARADOS (líneas 359–390) es análogo, pero asigna el punto al rival y deja el tiempo en cero, ya que no existe una reacción válida. En ambos casos la ISR solo levanta la bandera `resultado_pendiente`; no imprime nada.

8. Reporte del resultado y bucle principal

Fragmento 11. Copia protegida de los datos del resultado (líneas 511–528 de PICO_PRINCIPAL_EXAMEN.c)

```
// Mostrar automáticamente el resultado al terminar una partida
static void tarea_resultado(void) {

    // Si no hay resultado nuevo, no hacer nada
    if (!resultado_pendiente) {
        return;
    }

    // Copiar datos compartidos
    uint32_t estado_irq = save_and_disable_interrupts();
    estado_resultado_t r = resultado;
    uint8_t g = ganador;
    uint8_t jf = jugador_falso;
    uint64_t tiempo = tiempo_reaccion_us;
    uint32_t s1 = score_j1;
    uint32_t s2 = score_j2;

    resultado_pendiente = false;
    restore_interrupts(estado_irq);
}
```

tarea_resultado() revisa la bandera y, si hay un resultado nuevo, copia en variables locales todos los datos compartidos dentro de una sección crítica. Así se garantiza que el ganador, el tiempo de 64 bits y el marcador que se imprimen pertenecen a la misma partida, aunque una interrupción ocurra durante el formateo. El texto se arma con sprintf() fuera de la sección crítica, para mantener las interrupciones deshabilitadas el menor tiempo posible.

Fragmento 12. Bucle principal (líneas 739–742 de PICO_PRINCIPAL_EXAMEN.c)

```
while (true) {
    tarea_uart();
    tarea_resultado();
}
```

El bucle principal solo llama a dos tareas cortas que nunca esperan: si no hay bytes ni resultados pendientes, regresan de inmediato. Esta estructura es la que permite que las tres actividades del sistema ocurran a la vez.

9. Programa de la Pico de apoyo: puente USB–UART

Fragmento 13. Configuración de la Pico de apoyo (líneas 13–30 de PICO_APOYO_EXAMEN.c)

```
int main(void) {

    // Inicializa comunicación por usb
    stdio_init_all();

    // Configuración de los pines UART
    gpio_set_function(TX_PIN,GPIO_FUNC_UART);
    gpio_set_function(RX_PIN,GPIO_FUNC_UART);

    // Inicializa los pines como tipo UART
    uart_init(UART_ID,BAUD_RATE);

    // configurar formato UART
    uart_set_format(UART_ID,8,1,UART_PARITY_NONE);

    // Habilitar FIFO de la UART
    uart_set_fifo_enabled(UART_ID,true);
```

La Pico de apoyo inicializa stdio con `stdio_init_all()`; como en su `CMakeLists.txt` está activado `pico_enable_stdio_usb` y desactivado `pico_enable_stdio_uart`, la entrada y salida estándar viajan por USB y la tarjeta aparece en la computadora como `/dev/tty.usbmodem14401`, el puerto que se observa en las capturas del monitor serial. Después configura su UART0 en GP0 y GP1 con exactamente los mismos parámetros que la Pico principal (115200 baudios, 8N1 y FIFO habilitada); si alguno de estos valores fuera distinto, los bytes llegarían corruptos y se contarían como *framing errors*.

Fragmento 14. Bucle de reenvío en ambos sentidos (líneas 32–48 de `PICO_APOYO_EXAMEN.c`)

```
while (true) {

    // CONSOLA - UART APOYO - UART PRINCIPAL

    int ch = getchar_timeout_us(0);
    if (ch != PICO_ERROR_TIMEOUT) {
        uart_putc_raw(UART_ID,(char)ch);    // Llego caracter y lo manda a la pico principal
    }

    // UART PRINCIPAL - UART APOYO - CONSOLA
    while (uart_is_readable(UART_ID)) {

        // Leer un byte recibido desde la principal
        char c = uart_getc(UART_ID);
        putchar(c);    // Mandarlo a consola
    }
}
```

El bucle realiza dos tareas en cada vuelta. En el sentido computadora → Pico principal, `getchar_timeout_us(0)` revisa si llegó un carácter por USB sin esperar (el tiempo límite es cero); si lo hay, lo envía tal cual por la UART con `uart_putc_raw()`, que no hace conversiones de fin de línea, de modo que el `\r\n` del monitor llega intacto. En el sentido Pico principal → computadora, mientras `uart_is_readable()` indique que hay bytes en la FIFO de recepción, se leen con `uart_getc()` y se reenvían al USB con `putchar()`. Como ninguna de las dos operaciones bloquea, el puente atiende ambas direcciones de forma alternada y muy rápida;

a 115200 baudios llega un byte cada 87 μ s aproximadamente, mucho más tiempo del que tarda una vuelta del bucle, por lo que la FIFO de 32 bytes nunca se llena. El resultado es transparente: para la Pico principal es como si estuviera conectada directamente a la computadora.

Pruebas de funcionamiento por consola UART

Las pruebas se realizaron con el monitor serial de Visual Studio Code a 115200 baudios y terminación CRLF, conectado al puerto USB de la Pico de apoyo (/dev/tty.usbmodem14401); todo lo que aparece en las capturas pasó por el puente descrito en los Fragmentos 13 y 14. La Figura 5 muestra el mensaje de bienvenida que el sistema envía al arrancar.

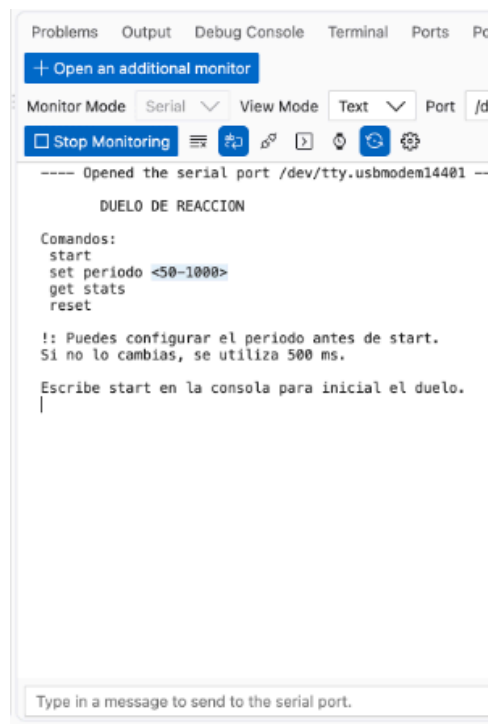


Figura 5. Mensaje de inicio del duelo de reacción en el monitor serial.

En la Figura 6 se prueba la validación de comandos: set periodo 30 (con error de escritura) se rechaza como comando desconocido, set periodo 30 se rechaza por estar fuera de rango y set periodo 350 se acepta. El comando get stats confirma el nuevo periodo, que aún no hay partidas y que no se han registrado errores de UART.

```

DUELO DE REACCION

Comandos:
start
set periodo <50-1000>
get stats
reset

!: Puedes configurar el periodo antes de start.
Si no lo cambias, se utiliza 500 ms.

Escribe start en la consola para inicial el duelo.
---- Sent utf8 encoded message: "set periodo 30\r\n" ----
set periodo 30Error: comando desconocido

---- Sent utf8 encoded message: "set periodo 30\r\n" ----
set periodo 30Error: periodo fuera de rango (50-1000 ms)

---- Sent utf8 encoded message: "set periodo 350\r\n" ----
set periodo 350Listo

---- Sent utf8 encoded message: "get stats\r\n" ----
get statsPeriodo configurado: 350 ms
Rango permitido: 50-1000 ms
Espera aleatoria: 2000-5000 ms
Ultima partida: Sin partidas | tiempo=N/A
UART: overruns=0 framing_errors=0 descartados=0
Score: J1 0, J2 0

Type in a message to send to the serial port.

```

Figura 6. Validación de comandos y consulta de estadísticas antes de la primera partida.

La Figura 7 muestra una partida completa con respuesta válida: el jugador 2 presionó primero con un tiempo de reacción de 558.393 ms, el marcador se actualizó a J1 0, J2 1 y get stats reporta el mismo resultado.

```

Problems  Output  Debug Console  Terminal  Ports  Polyglot

+ Open an additional monitor

Monitor Mode  Serial  View Mode  Text  Port  /dev/tty
[Stop Monitoring] [Icons]

---- Sent utf8 encoded message: "set periodo 350\r\n" ----
set periodo 350Listo

---- Sent utf8 encoded message: "get stats\r\n" ----
get statsPeriodo configurado: 350 ms
Rango permitido: 50-1000 ms
Espera aleatoria: 2000-5000 ms
Ultima partida: Sin partidas | tiempo=N/A
UART: overruns=0 framing_errors=0 descartados=0
Score: J1 0, J2 0

---- Sent utf8 encoded message: "start\r\n" ----
startListo

Ganador: J2 | Tiempo: 558.393 ms
Score: J1 0, J2 1
---- Sent utf8 encoded message: "get stats\r\n" ----
get statsPeriodo configurado: 350 ms
Rango permitido: 50-1000 ms
Espera aleatoria: 2000-5000 ms
Ultima partida: ganador=J2 tiempo=558.393 ms
UART: overruns=0 framing_errors=0 descartados=0
Score: J1 0, J2 1

```

Figura 7. Partida ganada por el jugador 2 con un tiempo de reacción de 558.393 ms.

En la Figura 8 se prueban los otros dos desenlaces: en la segunda partida el jugador 2 presionó durante el parpadeo rojo, por lo que se registró una salida en falso y el punto fue para el jugador 1; en la tercera nadie presionó y, al terminar la ventana, el sistema reportó "Sin ganador" sin modificar el marcador.

```

Score: J1 0, J2 1

---- Sent utf8 encoded message: "start\r\n" ----
startListo

Ganador: J1 | Salida en falso J2 | Tiempo: N/A
Score: J1 1, J2 1
---- Sent utf8 encoded message: "get stats\r\n" ----
get statsPeriodo configurado: 350 ms
Rango permitido: 50-1000 ms
Espera aleatoria: 2000-5000 ms
Ultima partida: ganador=J1 (salida en falso J2) tiempo=N/A
UART: overruns=0 framing_errors=0 descartados=0
Score: J1 1, J2 1

---- Sent utf8 encoded message: "start\r\n" ----
startListo

Sin ganador | Tiempo: N/A
Score: J1 1, J2 1
---- Sent utf8 encoded message: "get stats\r\n" ----
get statsPeriodo configurado: 350 ms
Rango permitido: 50-1000 ms
Espera aleatoria: 2000-5000 ms
Ultima partida: Sin ganador | tiempo=N/A
UART: overruns=0 framing_errors=0 descartados=0
Score: J1 1, J2 1

|
Type in a message to send to the serial port.

```

Figura 8. Salida en falso del jugador 2 y partida sin ganador.

Finalmente, la Figura 9 muestra que reset reinicia el marcador a cero sin borrar el periodo configurado ni el resultado de la última partida.

```

---- Sent utf8 encoded message: "reset\r\n" ----
resetListo

---- Sent utf8 encoded message: "get stats\r\n" ----
get statsPeriodo configurado: 350 ms
Rango permitido: 50-1000 ms
Espera aleatoria: 2000-5000 ms
Ultima partida: Sin ganador | tiempo=N/A
UART: overruns=0 framing_errors=0 descartados=0
Score: J1 0, J2 0

|
Type in a message to send to the serial port.

```

Figura 9. Reinicio del marcador con el comando reset.

Tabla 5. Resumen de las pruebas realizadas por consola.

Prueba	Resultado esperado	Resultado obtenido
--------	--------------------	--------------------

Comando mal escrito	Error: comando desconocido	Correcto
set periodo 30	Error por rango	Correcto
set periodo 350	Listo y periodo actualizado	Correcto
Respuesta con LED verde	Ganador y tiempo en ms	J2, 558.393 ms
Pulsación durante el rojo	Salida en falso, punto al rival	Falso J2, gana J1
Sin pulsación	Sin ganador tras la ventana	Correcto
reset	Marcador en 0	Correcto
Errores de UART	0 overruns, 0 framing, 0 descartados	0 / 0 / 0 en todas las consultas

En las capturas se observa que la respuesta de cada comando aparece en la misma línea que el eco (por ejemplo, startListo). Esto ocurre porque el programa procesa el comando al recibir `\r` y encola la respuesta antes del eco de `\n`, y el monitor no interpreta un `\r` aislado como salto de línea. No afecta al funcionamiento, pero podría corregirse enviando `\r\n` antes de cada respuesta.

Verificación del tiempo de reacción con osciloscopio

Para validar la medición se utilizó un osciloscopio Tektronix MDO3022. El canal 1 (amarillo) se conectó al pin GP17, que enciende el LED verde, y el canal 2 (azul) al pin del botón del jugador, con la referencia de ambas puntas en la tierra común. El disparo se configuró en el flanco de subida del canal 1, es decir, en el instante en que se enciende el LED verde. La base de tiempo fue de 400 ms/div con 10 000 puntos a 2.5 kS/s, lo que da una ventana de 4 s para capturar la partida completa y una resolución de 0.4 ms entre muestras.

Con esta conexión, el tiempo de reacción real es el intervalo entre el flanco de subida del canal 1 (el LED verde se enciende) y el flanco de bajada del canal 2 (el botón se presiona y, por el pull-up interno, el pin pasa de 3.3 V a 0 V). Es como cronometrar una carrera con dos fotos: una cuando se da la señal de salida y otra cuando el corredor cruza la meta. En las capturas, el canal 1 muestra un nivel de aproximadamente 0.31 V en lugar de 3.3 V; esto se debe a que la atenuación configurada en el canal no coincidía con la de la punta ($\times 10$), por lo que la amplitud aparece dividida entre diez. Este factor solo afecta la escala vertical y no modifica las mediciones de tiempo.

Primera medición: modo Roll

La primera partida se capturó en modo *Roll*, con el canal 1 a 20 mV/div (Figura 10). Con esa escala, cuando el LED verde se enciende la señal sale de la pantalla por la parte superior y solo vuelve a aparecer cuando el LED se apaga; justo en ese momento el canal 2 muestra el pulso bajo producido por el botón. Los cursores colocados en ambos extremos marcan $\Delta t = 216.0$ ms. Como los flancos del canal 1 quedan fuera de la pantalla, la posición de los cursores es menos precisa, por lo que se repitió la medición con una escala más adecuada.



Figura 10. Primera medición en modo Roll con un tiempo entre el encendido del LED verde y la pulsación de 216.0 ms.

Segunda medición: tiempo de reacción

En la segunda partida, el canal 1 se ajustó a 100 mV/div y la adquisición se detuvo después del evento. La Figura 11 muestra la vista completa: el canal 1 permanece en alto mientras el LED verde está encendido y regresa a cero exactamente cuando el canal 2 presenta el pulso bajo del botón. Esto confirma el comportamiento del Fragmento 10: la ISR del botón apaga el LED verde en cuanto detecta la pulsación válida.

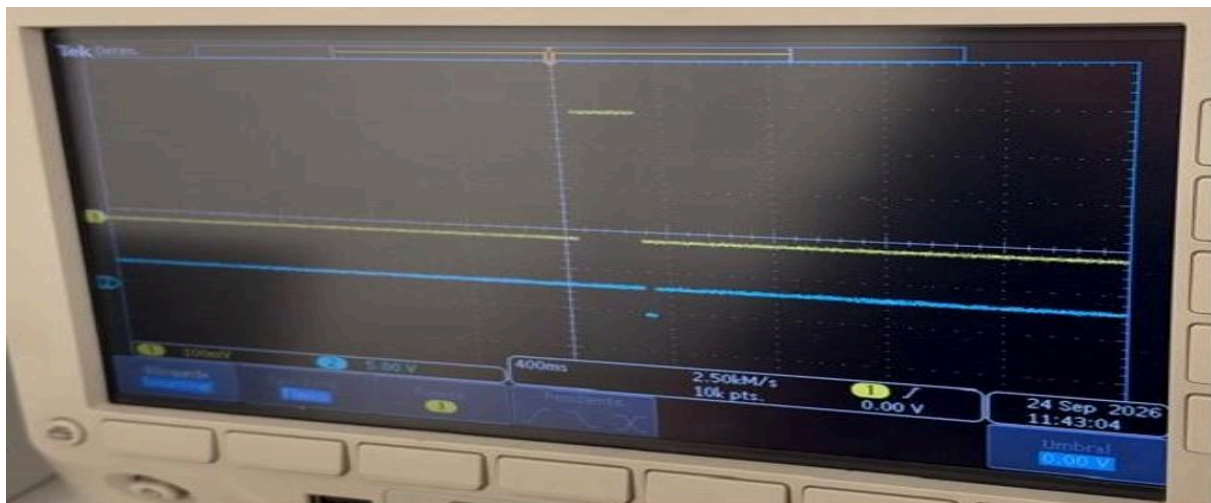


Figura 11. Vista completa de la segunda partida con el LED verde en el canal 1 y el botón en el canal 2.

Para medir con mayor precisión se utilizó la ampliación $\times 4$ (100 ms/div) centrada en el evento (Figura 12). El cursor **a** se colocó en el flanco de subida del LED verde (58.40 ms) y el cursor **b** en el flanco de bajada, que coincide con el inicio de la pulsación (302.4 ms). La diferencia entre ambos es $\Delta t = 244.0$ ms, que corresponde al tiempo de reacción del jugador en esta partida.



Figura 12. Medición del tiempo de reacción con cursores entre el encendido del LED verde y la pulsación, con $\Delta t = 244.0$ ms.

Duración de la pulsación

Sobre la misma adquisición, los cursores se movieron al canal 2 (Figura 13). El cursor **a** marca el flanco de bajada del botón (302.4 ms) y el cursor **b** su flanco de subida al soltarlo (345.4 ms), por lo que el botón permaneció presionado **43.0 ms**. Solo el flanco de bajada genera una interrupción (GPIO_IRQ_EDGE_FALL), así que la liberación del botón no provoca ningún evento en el programa. Además, la pulsación dura más que la ventana de antirrebote de 20 ms, por lo que cualquier rebote al presionar queda cubierto por ella. Con una resolución de 0.4 ms no es posible observar el rebote del contacto, que dura típicamente menos de un milisegundo; para verlo se necesitaría una base de tiempo del orden de microsegundos.



Figura 13. Medición de la duración de la pulsación en el canal 2 con $\Delta t = 43.0$ ms.

Comparación con la consola UART

Tabla 6. Comparación entre el tiempo medido por el osciloscopio.

Partida	Captura	Osciloscopio (ms)
1	Figura 10	216.0
2	Figuras 11 y 12	244.0

Ambos tiempos de reacción, 216.0 ms y 244.0 ms, se encuentran dentro del rango típico de la reacción humana ante un estímulo visual, de 200 a 300 ms aproximadamente. La precisión de la comparación está limitada por el osciloscopio y no por la Pico: con 2.5 kS/s cada muestra representa 0.4 ms y los cursores se colocan manualmente, por lo que la medición tiene una incertidumbre de alrededor de ± 1 ms. En cambio, las fuentes de error del programa (la instrucción entre `gpio_put(LED_VERDE, 1)` y la lectura de `t_inicio_verde`, la latencia de entrada a la ISR y la resolución de 1 μ s de `time_us_64()`) son de microsegundos. Por lo tanto, se espera que el valor reportado por la consola coincida con el del osciloscopio con una diferencia menor a 1 ms.

Conclusiones

Se implementó un duelo de reacción para dos jugadores en la Raspberry Pi Pico 2 que integra GPIO, interrupciones, un temporizador de hardware y una consola UART, cumpliendo con el requisito de que todas las funciones operen simultáneamente. La clave fue dividir el trabajo: las ISR solo registran eventos, toman marcas de tiempo y actualizan el estado, mientras que el bucle principal se encarga del texto y los comandos. Gracias a ello, el programa no utiliza ninguna espera bloqueante y la consola permanece disponible incluso durante una partida. La Pico de apoyo, por su parte, permitió usar la UART por hardware de

la Pico principal desde la computadora con un programa mínimo de reenvío, lo que separó la comunicación con el usuario de la lógica del juego.

La máquina de estados permitió que una misma pulsación se interpretara de forma distinta según el momento del juego, y reutilizar un solo timer repetitivo para el parpadeo, el tiempo aleatorio y la ventana de 5 s simplificó el diseño. Por su parte, los buffers circulares de recepción y transmisión desacoplaron la velocidad de la UART del resto del programa: en todas las pruebas se registraron cero *overruns*, cero errores de trama y cero bytes descartados.

Las pruebas por consola verificaron los tres desenlaces posibles (victoria con tiempo de reacción, salida en falso y partida sin ganador), la validación de comandos y el reinicio del marcador. Como mejoras se proponen: enviar un salto de línea antes de cada respuesta para separar el eco del texto, medir la ventana de 5 s con una marca de tiempo en lugar de un número de ejecuciones del timer para que no dependa del periodo configurado, y generar la semilla aleatoria a partir de un evento del usuario (por ejemplo, el instante en que se recibe start), ya que al arrancar el valor de `time_us_64()` es casi siempre el mismo.

Herramienta	Uso específico
ChatGPT	Apoyo en la escritura del código
Gemini	Apoyo en el código
Claude	Apoyo en la redacción y organización del documento del examen

Referencias

ARM Limited. (2007). *PrimeCell UART (PL011) technical reference manual* (Revisión r1p5, ARM DDI 0183G). <https://developer.arm.com/documentation/ddi0183/g>

freeCodeCamp. (2020, 6 de enero). *Finite state machine explained*.

freeCodeCamp.org. <https://www.freecodecamp.org/news/finite-state-machines/>

Ganssle, J. G. (2014). *A guide to debouncing, or, how to debounce a contact in two easy pages* (Rev. 4). The Ganssle Group. <https://www.ganssle.com/debouncing.htm>

h.larsen. (2023, 15 de mayo). *Use Pico's USB port as a UART for host communication* [Publicación en foro]. Raspberry Pi Forums.

<https://forums.raspberrypi.com/viewtopic.php?t=351506>

Jain, A., Bansal, R., Kumar, A., & Singh, K. D. (2015). A comparative study of visual and auditory reaction times on the basis of gender and physical activity levels of medical first year students. *International Journal of Applied and Basic Medical Research*, 5(2), 124–127. <https://doi.org/10.4103/2229-516X.157168>

Llamas, L. (s.f.). *Qué son y cómo usar los GPIO en Raspberry Pi*. Recuperado el 27 de septiembre de 2026, de <https://www.luisllamas.es/como-usar-gpio-raspberry-pi/>

Newbiely. (s.f.). *Raspberry Pi Pico – Button – Debounce*. Recuperado el 27 de septiembre de 2026, de

<https://newbiely.com/tutorials/raspberry-pico/raspberry-pi-pico-button-debounce>

Raspberry Pi Ltd. (2025). *RP2350 datasheet* (Build-version d126e9e-clean, 29 de julio de 2025). <https://datasheets.raspberrypi.com/rp2350/rp2350-datasheet.pdf>

Raspberry Pi Ltd. (2026a). *Getting started with Raspberry Pi Pico-series: C/C++ development with Raspberry Pi Pico-series and other Raspberry Pi microcontroller-based boards* (Release 24).

<https://datasheets.raspberrypi.com/pico/getting-started-with-pico.pdf>

Raspberry Pi Ltd. (2026b). *Raspberry Pi Pico 2 datasheet: An RP2350-based microcontroller board* (Release 5).

<https://datasheets.raspberrypi.com/pico/pico-2-datasheet.pdf>

Raspberry Pi Ltd. (2026c). *Raspberry Pi Pico-series C/C++ SDK: Libraries and tools for C/C++ development on Raspberry Pi microcontrollers* (Build-version ea68919-clean, 10 de septiembre de 2026).

<https://datasheets.raspberrypi.com/pico/raspberry-pi-pico-c-sdk.pdf>